Department of Computer Science and Engineering

CSA06 – Design and Analysis of Algorithms

CO1 – Assessment Tool 1: IT Application-Based Problem Solving

Scalability Audit of an E-Commerce Platform (“ShopSphere”)

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Mode	Individual
Student Reg. No.	192565006
Student Name	Thanigaivel S

(a) Understanding the Problem

The table gives the input size, the basic operation and the expected growth of each module. The basic operation is the step executed most often, so it decides the running time.

Module	Input size	Basic operation and where it occurs	Expected growth
M1 Duplicate detector	n = number of products	Title and seller comparison in the IF inside the inner loop.	Quadratic: every pair is compared, so n × 10 gives about 100 × work.
M2 Rate limiter	n = size of the request window	Audit(Request[j]) inside the inner FOR loop.	Linear: window sizes 1, 2, 4, … add up to about n.
M3 Search S1	n = products in an unsorted list	Comparison of the key with a product.	Linear: up to n comparisons.
M3 Search S2	n = entries in a sorted index	Comparison of the key with the middle entry.	Logarithmic: n is halved each time.
M4 Recommender	n = customer–product data size	Linear merge step (one operation per record) at each recursive call.	Current: quadratic. Redesign: n log n.
M5 Analytics	n = sales records	Comparison made while sorting at every recursion level.	About n log² n.
Report generator	n = products	Operations counted by f(n) = 3n² + 10n log n + 50.	Quadratic: 3n² dominates.

Brief justification

- **M1:** Two nested loops compare each product i with every later product j. The comparison is the most repeated step. The number of pairs grows like n^2.
- **M2:** The inner loop runs i times and i doubles each round. Audit is the only real work, and the total is a doubling (geometric) sum, so it grows like n, not n log n.
- **M3:** S1 may check every product. S2 throws away half of the index after each comparison, so it needs only about log n comparisons.
- **M4:** Each call does linear merge work and calls itself on smaller parts. Four calls of size n/2 make the work pile up (n^2). Two calls of size n/2 keep it at n log n.
- **M5:** Sorting n records at the top level costs about n log n, and every lower level repeats that sorting work on smaller pieces. This adds an extra log n factor.
- **Report generator:** The n^2 term is the largest term for big n. The n log n and constant terms matter very little.

(b) Analysis of the Iterative Modules

Module M1 – Duplicate-Listing Detector

Step 1. For a fixed i, the inner loop j runs from i + 1 to n − 1. The number of values of j is:

$$(n - 1) - (i + 1) + 1 = n - 1 - i$$

Step 2. The IF test (the basic operation) runs once for each j. The outer loop runs i = 0 to n − 2. So the total number of comparisons is:

$$C(n) = \Sigma\ (n - 1 - i), \text{ for } i = 0 \text{ to } n - 2$$

Step 3. Writing out the terms:

$$C(n) = (n - 1) + (n - 2) + \ldots + 2 + 1$$

Step 4. This is the sum of the first (n − 1) natural numbers, which equals (n − 1)·n / 2:

$$\mathbf{C(n) = n(n - 1) / 2 = (n^2 - n) / 2}$$

This count is the same for every input, because the loops do not stop early. So the best, average and worst cases are all equal.

Time complexity: $C(n) = \frac{1}{2}n^2 - \frac{1}{2}n$. The leading term is $\frac{1}{2}n^2$, so the time is **$\Theta(n^2)$**.

Space complexity: Only the loop counters i and j are used apart from the input. The extra space is **$\Theta(1)$**.

Check: n = 4 gives 3 + 2 + 1 = 6 and 4 × 3 / 2 = 6. ✓

Meaning for ShopSphere: at $n = 10^5$ there are about 5×10^9 comparisons (50 s). At $n = 10^6$ there are about 5×10^{11} comparisons (5,000 s).

Module M2 – API Rate Limiter

Step 1. The value of i at each round of the WHILE loop is 1, 2, 4, 8, …, that is 2^0, 2^1, 2^2, … The loop continues while i < n.

Step 2. Let k be the number of rounds. The last value used is 2^(k−1), which must be less than n. So $k = \lceil \log_2 n \rceil$. For n = 2^m this gives k = m.

Step 3. In the round where the value is i, the FOR loop runs from j = 0 to i − 1, which is exactly i Audit operations. So the total is:

$$A(n) = 2^0 + 2^1 + 2^2 + \ldots + 2^{(k-1)}$$

Step 4. This is a geometric series with ratio 2:

$$\mathbf{A(n) = 2^k - 1}$$

Step 5. Since $k = \lceil \log_2 n \rceil$, we have $n \le 2^k < 2n$. Therefore:

$$n - 1 \le A(n) < 2n - 1$$

When n is a power of 2, the exact answer is **$A(n) = n - 1$**.

Is the team member's claim Θ(n log n) correct? No. The member counted the inner loop as n operations in every round. But the inner loop runs only i times, and i is small in the early rounds (1, 2, 4, …). Only the last round has a large loop. The product (log n) × n is a valid upper bound, but it is not tight. The tight bound is:

$$\mathbf{A(n) = \Theta(n)}$$

Reason: $A(n) \ge n - 1 \ge \frac{1}{2}n$ for $n \ge 2$, and $A(n) < 2n$. So $\frac{1}{2}n \le A(n) \le 2n$.

Verification for n = 1,024: the values of i are 1, 2, 4, 8, 16, 32, 64, 128, 256, 512. The next value 1,024 is not less than n, so the loop stops. That is k = 10 rounds.

$$A(1024) = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 1{,}023$$

Formula check: $2^{10} - 1 = 1{,}023 = n - 1$. ✓ The claimed n log n would give 1,024 × 10 = 10,240, which is about 10 times too large. So the claim is wrong.

Space complexity: Θ(1). **Meaning:** for $n = 10^6$, $A(n) = 2^{20} - 1 = 1{,}048{,}575 \approx 10^6$ operations, which is only about 0.01 s.

(c) Recurrence by Substitution – Search Version S2

Given: T(n) = T(n/2) + 1 and T(1) = 1. Assume n = 2^k so that n/2 is always a whole number.

Backward substitution, step by step

$$T(n) = T(n/2) + 1$$

Replace T(n/2) using the same rule, T(n/2) = T(n/4) + 1:

$$T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2$$

Replace T(n/4) = T(n/8) + 1:

$$T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3$$

Pattern after i substitutions:

$$T(n) = T(n/2^i) + i$$

The recursion stops when n/2^i = 1, that is 2^i = n, so $i = \log_2 n$. Putting $i = \log_2 n$ and using T(1) = 1:

$$T(n) = T(1) + \log_2 n$$

$$\mathbf{T(n) = 1 + \log_2 n}$$

Check: T(8) = T(4) + 1 = T(2) + 2 = T(1) + 3 = 4, and $1 + \log_2 8 = 4$. ✓

For any n (not only powers of 2), the exact form is $\lfloor \log_2 n \rfloor + 1$.

Time and space complexity

Version	Time complexity	Space complexity
Iterative binary search	Θ(log n) (best case Θ(1))	Θ(1): only low, high and mid are stored.
Recursive binary search	Θ(log n) (best case Θ(1))	Θ(log n): the call stack holds up to $1 + \log_2 n$ frames.
S1 sequential scan (for comparison)	Θ(n) worst case	Θ(1)

Meaning for ShopSphere: at $n = 10^6$, S2 needs only about 1 + 19.93 ≈ 21 comparisons per search, while S1 may need 10^6. Both versions take the same time,

but the iterative one uses less memory. The price of S2 is that the index must stay sorted.

(d) Master Theorem

For T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a):

- Case 1: f(n) = O(n^(log_b a − ε)) for some ε > 0, then T(n) = Θ(n^(log_b a)).
- Case 2: f(n) = Θ(n^(log_b a)), then T(n) = Θ(n^(log_b a) · log n).
- Case 3: f(n) = Ω(n^(log_b a + ε)) and the regularity condition holds, then T(n) = Θ(f(n)).

Module M4 – Recommendation Engine

Item	**Current design: T(n) = 4T(n/2) + n**	**Redesign: T(n) = 2T(n/2) + n**
a	4	2
b	2	2
f(n)	n	n
n^(log_b a)	$n^{(\log_2 4)} = n^2$	$n^{(\log_2 2)} = n^1 = n$
Comparison	f(n) = n = O(n^(2−ε)) with ε = 1	f(n) = Θ(n) = Θ(n^(log_b a))
Case used	Case 1	Case 2
Result	**$T(n) = \Theta(n^2)$**	**$T(n) = \Theta(n \log n)$**

What happens when the data doubles

- **Current design (n^2):** $T(2n) / T(n) = (2n)^2 / n^2 =$ **4**. Doubling the data makes the time 4 times longer.
- **Redesign (n log n):** $T(2n) / T(n) = 2n \cdot \log(2n) / (n \cdot \log n) = 2(1 + 1/\log n)$. For $n = 10^6$ this is $2 \times (1 + 1/19.93) \approx$ **2.1**. Doubling the data makes the time just a little more than double.

Growth from 10^5 to 10^6: the current design becomes 100 times slower, but the redesign becomes only about 12 times slower ($1.99 \times 10^7 / 1.66 \times 10^6$). The redesign scales much better.

Module M5 – Sales Analytics Engine: T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, f(n) = n log n and n^(log_b a) = n.

- **Case 1:** needs f(n) to be polynomially smaller than n. But n log n is larger than n. Not applicable.
- **Case 2:** needs f(n) = Θ(n). But n log n grows faster than n by a log n factor. Not applicable.
- **Case 3:** needs f(n) = Ω(n^(1+ε)) for some ε > 0. But log n grows slower than any n^ε, so n log n is not polynomially larger than n. Not applicable.

None of the three standard cases applies. The function is in the gap between Case 2 and Case 3. We use the extended Master Theorem.

Extended Master Theorem: if f(n) = Θ(n^(log_b a) · log^k n) with k ≥ 0, then T(n) = Θ(n^(log_b a) · log^(k+1) n). Here k = 1, so:

$$\mathbf{T(n) = \Theta(n \log^2 n)}$$

Verification by recursion tree

- Level i has 2^i subproblems, each of size n/2^i.
- Cost of one subproblem at level i = (n/2^i) · log(n/2^i) = (n/2^i)(log n − i).
- Cost of the whole level i = 2^i × (n/2^i)(log n − i) = n(log n − i).
- There are log n levels (i = 0 to log n − 1) plus a leaf level costing Θ(n).

$$\text{Total} = n \cdot \Sigma\,(\log n - i) + \Theta(n) = n \cdot [\log n + (\log n - 1) + \ldots + 1] + \Theta(n)$$

$$= n \cdot \log n\,(\log n + 1) / 2 + \Theta(n) = \mathbf{\Theta(n \log^2 n)}$$

Meaning: M5 is only a log n factor (about 20 times) slower than a plain n log n algorithm, but it is far faster than any n² algorithm.

(e) Best, Average and Worst Cases

Search S1 – Sequential scan (unsorted)

Case	Input condition	Comparisons
Best	The searched product is the first item.	1, i.e. Θ(1)
Average	The product is at a random position (or may be missing).	About (n + 1)/2 if present; see below
Worst	The product is the last item or is not in the list.	n, i.e. Θ(n)

Average case of S1 with probability p

Let p be the probability that the product exists. If it exists, it is equally likely to be at any of the n positions, each with probability p/n.

- Found at position i: i comparisons, probability p/n.
- Not found: n comparisons, probability (1 − p).

$$A(n) = \Sigma\, [i \cdot p/n] + n(1 - p), \text{ for } i = 1 \text{ to } n$$

$$= (p/n) \cdot n(n + 1)/2 + n(1 - p)$$

$$\mathbf{A(n) = p(n + 1)/2 + n(1 - p) = [(2 - p)n + p] / 2}$$

Special cases: p = 1 gives (n + 1)/2, and p = 0 gives n. ✓

For n = 1,000,000 and p = 0.5:

$$A = 0.5 \times (1{,}000{,}001) / 2 + 1{,}000{,}000 \times 0.5$$

$$= 250{,}000.25 + 500{,}000 = \mathbf{750{,}000.25 \approx 7.5 \times 10^5 \text{ comparisons}}$$

At 10^8 operations per second, this is about 0.0075 s per search.

Search S2 – Binary search (sorted index)

Case	Input condition	Comparisons
Best	The key is the middle entry of the index.	1, i.e. Θ(1)

Case	Input condition	Comparisons
Average	The key is at a random position (found after about log n − 1 halvings).	About $\log_2 n - 1 \approx 19$ for $n = 10^6$, i.e. $\Theta(\log n)$
Worst	The key is not present, or is found only when one entry is left.	$1 + \log_2 n \approx 21$ by the recurrence ($\lfloor \log_2 n \rfloor + 1 = 20$ for $n = 10^6$ exactly), i.e. $\Theta(\log n)$

Meaning: even the worst case of S2 is tiny compared with the average case of S1 (21 compared with 750,000). S2 is the right choice for ShopSphere, as long as the index is kept sorted.

(f) Proving an Asymptotic Bound

Report generator cost: $f(n) = 3n^2 + 10n \log_2 n + 50$.

Definition (Θ): $f(n) = \Theta(g(n))$ if there are constants $c_1, c_2 > 0$ and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$. Here $g(n) = n^2$.

Lower bound (c_1)

For $n \geq 1$, the terms $10n \log_2 n$ and 50 are not negative. So:

$$f(n) = 3n^2 + 10n \log_2 n + 50 \geq 3n^2$$

So $c_1 = 3$ works for all $n \geq 1$.

Upper bound (c_2)

For $n \geq 1$ we have $\log_2 n \leq n$, so $10n \log_2 n \leq 10n^2$. Also $50 \leq 50n^2$. Therefore:

$$f(n) \leq 3n^2 + 10n^2 + 50n^2 = 63n^2$$

So $c_2 = 63$ works for all $n \geq 1$.

Conclusion

$$3n^2 \leq f(n) \leq 63n^2, \text{ for all } n \geq n_0 = 1$$

With **$c_1 = 3$, $c_2 = 63$ and $n_0 = 1$**, the definition is satisfied. Hence **$f(n) = \Theta(n^2)$**.

Check: $n = 1$ gives $f = 53$, and $3 \leq 53 \leq 63$. ✓ $n = 10$ gives $f = 300 + 332 + 50 = 682$, and $300 \leq 682 \leq 6{,}300$. ✓

Proof that f(n) is not O(n)

Definition (O): $f(n) = O(n)$ if there are constants $c > 0$ and n_0 such that $f(n) \leq c \cdot n$ for all $n \geq n_0$.

Suppose, for contradiction, that $f(n) = O(n)$. Then for some c and n_0:

$$3n^2 \leq f(n) \leq c \cdot n, \text{ for all } n \geq n_0$$

Dividing by n ($n > 0$) gives $3n \leq c$, so $n \leq c/3$ for every $n \geq n_0$. But n can be made as large as we like. Choose $n = \max(n_0, \lceil c/3 \rceil + 1)$. Then $n > c/3$, which contradicts $n \leq c/3$.

So no such c and n_0 exist. Therefore **f(n) is not O(n)**. ■

Meaning: the report generator is truly quadratic. It cannot be made to behave like a linear algorithm by tuning constants.

(g) Scalability Decision

Window limit: 10 minutes = 600 s = 6×10^{10} operations. Time = operations / 10^8.

Estimates for n = 1,000,000

Module	Complexity	Operations	Time	Meets 10 min?
M1	$n(n-1)/2$	$\approx 5 \times 10^{11}$	5,000 s ≈ 83.3 min	**No**
M2	2^k − 1 = Θ(n)	1,048,575 ≈ 1.05×10^6	≈ 0.0105 s	Yes
S2	$1 + \log_2 n$	≈ 21 per search	$\approx 2 \times 10^{-7}$ s	Yes
M4 current	$\Theta(n^2)$	$\approx 10^{12}$	10,000 s ≈ 2.8 h	**No**
M4 redesign	$\Theta(n \log n)$	$\approx 2 \times 10^7$	≈ 0.2 s	Yes
M5	$\Theta(n \log^2 n)$	$\approx 4 \times 10^8$	≈ 4 s	Yes
Report generator	$3n^2 + \ldots$	$\approx 3 \times 10^{12}$	≈ 30,000 s ≈ 8.3 h	**No**

Why this matters now: at the present size $n = 10^5$, M1 takes about 50 s, the current M4 takes about 100 s and the report generator about 300 s. All fit in the window today. After a 10× growth they become 100× slower and fail.

Modules that fail the 10-minute window

- **M1 – Duplicate-Listing Detector** (about 83 minutes).
- **M4 current design** (about 2.8 hours).
- **Report generator** (about 8.3 hours).

M2, S2, the redesigned M4 and M5 all finish well inside the window.

Recommendations

- **Redesign M1:** put each (seller, title) pair in a hash set and check it once per product. This takes expected $\Theta(n)$ time and $\Theta(n)$ space, about 10^6 operations, which is under 0.02 s. Another option is to sort by (seller, title) and compare neighbours, which takes $\Theta(n \log n)$. A check at upload time would also avoid the nightly job.
- **Adopt the M4 redesign:** removing the replicated partitions changes the recurrence from $4T(n/2) + n$ to $2T(n/2) + n$, which reduces $\Theta(n^2)$ to $\Theta(n \log n)$. This is a gain of about 50,000 times at $n = 10^6$.
- **Redesign the report generator:** find the part that works on pairs of items (the $3n^2$ term) and replace it with sorting or hash-based grouping. This brings the cost down to $\Theta(n \log n)$.
- **Keep M2:** it is already linear ($\Theta(n)$), so no change is needed. The team's $\Theta(n \log n)$ estimate was wrong, so it needs no extra capacity.
- **Use S2 for searching:** keep the product index sorted and use binary search. S1 costs about 7.5×10^5 comparisons on average, but S2 needs only about 20.
- **Keep M5, with an optional improvement:** it finishes in about 4 s. If needed, sort the data once and only merge at each level. That changes the recurrence to $2T(n/2) + n$, which gives $\Theta(n \log n)$.

Overall Conclusion

The audit shows that the algorithm's growth rate matters more than the speed of the servers. Two modules and the report generator are quadratic, so they will fail when the catalogue grows from 10^5 to 10^6 products. The fixes (hashing or sorting for M1, the replication-free design for M4, and a better grouping method for the report generator) bring them to $\Theta(n)$ or $\Theta(n \log n)$. With these changes, every module will finish well inside the 10-minute nightly window.